Name	Likith Kumar
Email ID	likithkumarg425@gmail.com
Course Code	102505
Assignment Topic	Advanced Monitoring, Logging, Alerting & Distributed Tracing
Source Code Provided	YES
GIT URL	https://github.com/LikithKumar0112/monitoring-capstone.git

Private & Confidential – Do not duplicate or distribute without written permission"

Identified Mistakes and Areas for Improvement Advanced Monitoring, Logging, Alerting & Distributed Tracing – Assignment Review

Problem Statement

The project implements a complete end-to-end observability platform across four key stages:

- Prometheus + Grafana for infrastructure and container monitoring
- ELK Stack for centralized application logging and analysis
- Alertmanager + Jaeger for alerting and distributed tracing
- Thanos + MinIO for scalable, highly available, and durable monitoring

The project also includes a unified Docker Compose platform with profiles, automated Grafana dashboard provisioning, smoke-test validation, documentation, AWS EC2 deployment guidance, and runtime screenshots.

Overall, the implementation substantially covers the expected monitoring and observability requirements. A key strength is the inclusion of practical validation evidence for Prometheus targets, Grafana dashboards, Kibana logs, Slack alerts, Jaeger traces, and Thanos components.

Key Observations

1. Prometheus and Grafana Monitoring

Issue Identified

The Prometheus and Grafana implementation is well structured and provides both infrastructure-level and container-level monitoring.

Observed Behavior

Prometheus is configured to collect metrics from:

- Prometheus itself
- Node Exporter
- cAdvisor

Node Exporter is configured with host filesystem access using:

Private & Confidential – Do not duplicate or distribute without written permission"

.terraform.lock.hcl

and the host root filesystem is mounted read-only.

cAdvisor is configured to collect Docker container metrics and exposes port 8080.

Prometheus retains metrics for 15 days and uses a persistent Docker volume for its TSDB.

Grafana is provisioned automatically with:

- Prometheus datasource
- Node Exporter dashboard
- Custom host dashboard

The repository also contains a custom HighCpuUsage alert rule.

Opportunity for Enhancement

The current 5.1 monitoring stack primarily focuses on infrastructure and container metrics. Application-level metrics are not part of this stage.

The cAdvisor configuration also uses:
privileged: true

and mounts several host-level paths.

Why This Matters

Host and container metrics provide good infrastructure visibility, but application-level metrics are important when moving toward full observability.

The privileged cAdvisor configuration is useful for a lab environment, but it increases the container's access to the host.

Key Findings

- Prometheus is correctly configured.
- Node Exporter provides host metrics.
- cAdvisor provides container metrics.
- Prometheus has persistent metric storage.
- 15-day retention is configured.

Private & Confidential – Do not duplicate or distribute without written permission"

- Grafana datasource provisioning is automated.
- Dashboards are provisioned as code.
- A custom CPU alert is implemented.
- Image versions are pinned.
- Runtime evidence shows Prometheus targets are UP.
- Grafana dashboards contain actual monitoring data.

Suggested Improvements

For a production-oriented implementation, reduce privileged access where practical and add application-specific metrics alongside infrastructure metrics. The custom dashboard could also include explicit alert thresholds and service-level indicators.

2. Centralized Logging with ELK

Issue Identified

The ELK implementation provides a complete log-processing pipeline and demonstrates structured log parsing rather than simply forwarding raw log messages.

Observed Behavior

The logging flow is:

Log Generator → Filebeat → Logstash → Elasticsearch → Kibana

The log generator writes application logs to a shared volume.

Filebeat tails:

```
/logs/app.log
```

and sends the events to Logstash on port 5044.

Logstash uses Grok to extract:

- Timestamp
- Log level
- Service
- Message

The date filter then converts the extracted timestamp into Elasticsearch's @timestamp.

Private & Confidential – Do not duplicate or distribute without written permission"

Elasticsearch stores the events in daily indexes using:

```
app-logs-*
```

Kibana is configured to search the resulting logs.

Opportunity for Enhancement

Elasticsearch security is explicitly disabled:

```
xpack.security.enabled=false
```

The project correctly identifies this as a demo configuration, but the production security implementation is not included.

Why This Matters

The current configuration is suitable for a controlled learning environment. However, exposing Elasticsearch and Kibana without authentication would not be appropriate for a production deployment.

Key Findings

- Filebeat log collection is implemented.
- Logstash receives Beats input.
- Grok parsing is implemented.
- Event timestamp is extracted from the application log.
- @timestamp is populated using the date filter.
- Elasticsearch uses persistent storage.
- Kibana provides log search and visualization.
- Logstash and Elasticsearch healthchecks are implemented.
- Elasticsearch security is disabled for the demonstration.
- Runtime screenshots show parsed logs in Kibana.

Suggested Improvements

Enable Elasticsearch security for a production deployment and configure authenticated communication between Elastic components. Additional metadata such as environment, application name, and request ID could also be added to improve troubleshooting.

Private & Confidential – Do not duplicate or distribute without written permission"

3. Advanced Alerting with Prometheus and Alertmanager

Issue Identified

The project goes beyond defining alert rules and demonstrates an actual alert lifecycle from Prometheus to Alertmanager and then to Slack.

Observed Behavior

Two important alerting rules are implemented:

InstanceDown

Detects unavailable scrape targets using:

```
up == 0
```

and requires the condition to remain true for one minute.

HighErrorRate

Uses the application's recorded error ratio and fires when the 5xx rate remains above 10% for two minutes.

The recording rule calculates the application's error ratio first, allowing the alert to operate on the precomputed metric.

Alertmanager groups alerts by alertname and instance, sends notifications to Slack, and is configured to send resolved notifications.

An inhibition rule is also configured so critical alerts can suppress warning alerts for the same instance.

Opportunity for Enhancement

The alerting implementation is strong, but the inhibition behavior is configured rather than demonstrated in the submitted project evidence

Why This Matters

Configuration demonstrates intended behavior, while an actual inhibition test would confirm that the routing policy works as expected.

Key Findings

Private & Confidential – Do not duplicate or distribute without written permission"

- Recording rules are implemented.
- Alerting rules are implemented.
- for durations are configured.
- Alert severity labels are used.
- Alertmanager routing is configured.
- Slack notification integration is implemented.
- Resolved notifications are enabled.
- Inhibition rules are configured.
- Runtime evidence shows HighErrorRate firing.
- Slack notification evidence is included.

Suggested Improvements

Demonstrate the inhibition rule with a controlled test where a critical alert is active and a lower-severity alert for the same instance is suppressed. This would provide stronger evidence of Alertmanager's advanced routing behavior.

4. Distributed Tracing with Jaeger and HotROD

Issue Identified

The project successfully demonstrates distributed tracing with actual trace data rather than only deploying Jaeger.

Observed Behavior

Jaeger is deployed using the all-in-one image with OTLP enabled.

HotROD sends trace data to Jaeger through:

```
http://jaeger:4318
```

The submitted evidence shows a frontend /dispatch trace containing multiple services and spans.

Prometheus also scrapes Jaeger's own metrics through port 14269.

Opportunity for Enhancement

The tracing implementation uses the standard HotROD example

Private & Confidential – Do not duplicate or distribute without written permission"

application, so the project does not demonstrate custom instrumentation or application-specific span enrichment.

Why This Matters

HotROD is useful for demonstrating distributed tracing concepts, but custom application instrumentation would show how the same approach applies to a real application.

Key Findings

- Jaeger is correctly deployed.
- OTLP is enabled.
- HotROD generates trace traffic.
- Jaeger receives the traces.
- Multi-service spans are visible.
- Jaeger metrics are integrated with Prometheus.
- Runtime trace evidence is provided.

Suggested Improvements

Add one custom span or tag to the application and demonstrate it in Jaeger. This would strengthen the "Beyond the Basics" portion and demonstrate practical instrumentation knowledge.

5. Thanos-Based Monitoring Scalability

Issue Identified

The Thanos implementation demonstrates a meaningful multi-Prometheus architecture rather than simply running multiple Prometheus containers.

Observed Behavior

Two leaf Prometheus instances are configured:

- prometheus-us
- prometheus-eu

They use different external labels:

```
region: us-east
region: eu-west
```

Both also use:

```
replica: A
```

Each Prometheus has a Thanos Sidecar that uploads TSDB blocks to MinIO.

The architecture also includes:

- Thanos Store
- Thanos Compact
- Thanos Query
- MinIO object storage
- Grafana connected to Thanos Query

Thanos Query exposes the global Prometheus-compatible query API.

The submitted evidence shows both regional stores connected and Grafana querying data from both regions.

Opportunity for Enhancement

The project includes a federation Prometheus instance, but it is intentionally positioned as a comparison to Thanos rather than part of the Thanos data path.

This distinction is documented well, but the practical limitations of federation could be demonstrated more explicitly.

Why This Matters

The important architectural concept is that Thanos provides a global query layer and durable object-storage-backed history without requiring a single Prometheus instance to hold all data.

Key Findings

- Multiple Prometheus instances are implemented.
- External labels distinguish regions.
- Thanos Sidecars are configured.
- MinIO is used as S3-compatible object storage.
- Thanos Store is implemented.
- Thanos Compact is implemented.

Private & Confidential – Do not duplicate or distribute without written permission"

- Thanos Query provides the global query endpoint.
- Grafana uses Thanos Query as its datasource.
- Cross-region querying is demonstrated.
- Federation is included as a comparison architecture.

Suggested Improvements

Demonstrate historical querying after local Prometheus retention has expired and show Thanos Compact/downsampling behavior with older blocks. This would provide stronger evidence of the long-term storage capability.

6. Unified End-to-End Observability Platform

Issue Identified

The unified e2e implementation is a strong addition because it combines the individual projects into one profile-based platform.

Observed Behavior

The platform supports profiles for:

- Metrics
- Logging
- Alerting
- Tracing
- Scaling
- All components together

The design uses a single Grafana instance and provisions multiple datasources and dashboards.

The project also includes:

- .env.example
- Makefile
- verify.sh
- Healthchecks
- Memory limits
- Persistent volumes
- Profile-specific startup

Private & Confidential – Do not duplicate or distribute without written permission"

The verify.sh script checks:

- Containers
- Prometheus targets
- Recording/alerting rules
- Alertmanager
- Elasticsearch health
- Log index document count
- Grafana health
- Thanos stores
- Jaeger traces

Opportunity for Enhancement

The all profile is documented at approximately 7.6 GB of memory caps on an 8 GB EC2 instance.
This leaves very little operating-system headroom.

Why This Matters

The platform may technically fit within the stated hardware limits, but simultaneous execution of all components leaves little room for Docker overhead, the operating system, and temporary workload spikes.

Key Findings

- Unified Compose architecture is implemented.
- Compose profiles control resource usage.
- Memory limits are explicitly configured.
- Persistent volumes are used.
- Healthchecks are implemented across important services.
- Automated smoke testing is included.
- The verification script checks multiple layers of the observability platform.
- The all profile is resource-constrained on the recommended 8 GB host.

Suggested Improvements

Continue recommending profile-specific execution for normal

Private & Confidential – Do not duplicate or distribute without written permission"

development. If the complete all profile is required regularly, increase the EC2 memory capacity and document the expected resource headroom.

7. Security, Secrets, and Production Readiness

Issue Identified

The project demonstrates good awareness of secret handling in the unified stack, but the individual 5.4 implementation still contains development credentials directly in its configuration.

Observed Behavior

The root project documentation states that secrets are stored in a git-ignored .env file, including:

- Grafana password
- MinIO password
- Slack webhook
- Elasticsearch password

The unified e2e configuration references environment variables.

However, the standalone 5.4 configuration contains:

```
MINIO_ROOT_USER: minioadmin
MINIO_ROOT_PASSWORD: minioadmin
```

and the Thanos object-store configuration also contains the same credentials.

The 5.4 object store additionally uses:

```
insecure: true
```

because MinIO is running over HTTP internally.

Opportunity for Enhancement

There is therefore a difference between the security approach documented for the unified platform and the standalone 5.4 implementation.

Elasticsearch security is also disabled in the 5.2 and e2e logging

configuration.

Grafana defaults to admin/admin when environment variables are not supplied.

Why This Matters

These defaults are acceptable for a controlled lab, but they should not be carried into a production environment or exposed through an externally accessible security group.

Key Findings

- .env-based secret handling is implemented for the unified stack.
- .env is git-ignored.
- Slack webhook is injected at runtime.
- Standalone 5.4 contains hardcoded MinIO development credentials.
- Thanos object storage uses HTTP.
- Elasticsearch security is disabled.
- Grafana has development defaults.
- AWS setup documentation correctly recommends restricting dashboard ports to My IP rather than 0.0.0.0/0.

Suggested Improvements

Apply the same environment-variable/secret strategy consistently to standalone 5.4. For production, use proper secret management, TLS, authenticated Elasticsearch, and non-default credentials.

8. Documentation, Validation, and Overall Implementation Issue Identified

The project is extensively documented and includes considerably more validation evidence than a basic monitoring submission.

Observed Behavior

The repository contains:

- Main README

Private & Confidential – Do not duplicate or distribute without written permission"

- Architecture documentation
- AWS setup guide
- Build guide
- Deliverables checklist
- Individual project READMEs
- E2E README
- Screenshots
- Automated smoke-test script

The documentation clearly maps the four projects to their required deliverables.

The screenshots demonstrate:

- Prometheus targets
- Grafana datasource
- Grafana dashboards
- cAdvisor metrics
- Kibana logs
- Logstash pipeline
- Prometheus rules
- Firing alerts
- Slack notification
- Jaeger traces
- Thanos stores
- Cross-region Grafana queries

Opportunity for Enhancement

The documentation is very comprehensive, but some production-oriented capabilities remain explicitly identified as stretch goals rather than implemented features.

Examples include:

- Elasticsearch authentication
- Thanos downsampling validation
- Multi-tenancy
- HA Grafana
- Custom tracing instrumentation

Private & Confidential – Do not duplicate or distribute without written permission"

- Alertmanager inhibition demonstration

Why This Matters

Clearly separating implemented functionality, demonstrated functionality, and planned/stretch functionality makes a technical capstone easier to evaluate and maintain.

Key Findings

- Documentation is detailed and technically organized.
- Required deliverables are clearly identified.
- Runtime screenshots support the major implementation claims.
- Automated E2E validation is included.
- Architecture documentation explains component responsibilities and failure modes.
- Stretch goals are clearly distinguished.
- The project provides both individual stacks and a unified platform.

Suggested Improvements

Maintain a final implementation checklist indicating which required and stretch features are implemented, tested, and demonstrated. This would make the final submission easier for an instructor or reviewer to validate.

Private & Confidential – Do not duplicate or distribute without written permission"